



SLT WhatsApp Agent

Sri Lanka Bureau of Foreign Employment (SLBFE)

Project Setup & Usage Manual (Laravel)

Version: 1.0

Table of Contents

1. Laravel and MVC Architecture.....	5
1.1 Introduction.....	5
1.2 What is Laravel?	5
1.3 What is MVC Architecture?.....	5
1.4 Laravel and MVC Architecture	6
1.5 Advantages of Laravel with MVC.....	6
2. Project overview and purpose.....	7
3. Key features.....	7
4. Overall System Flow	8
5. Requirements.....	9
6. Functional Flow of the System	9
6.1 Login Flow	9
6.2 Chat List Flow	9
6.3 Message Retrieval Flow	9
6.4 Sending Reply Flow.....	9
6.5 Contact Synchronization Flow	9
6.6 Log Monitoring Flow	9
7. Required Files to Run the Project.....	10
8. Controllers of the Project	11
8.1. ChatController.....	11
8.2. ChatApiController	12
8.3. ChatLockController	13
8.4. ContactController	14
8.5. LogController.....	15
8.6. ProfileController	16
8.7. Authentication Controllers	17
8.9. Common Errors in Controllers and Their Solutions	18
9. Models of the Project	19
9.1. Contact Model.....	19
9.2. Message Model.....	21
9.3. ApiLog Model	23

9.4. User Model.....	25
9.5. Relationships Between Models.....	26
9.6. Common Errors in Models and Their Solutions	26
10. Technical Architecture	28
10.1. Route Files	28
10.2. Blade View Files.....	29
10.3. Service Layer	30
10.4. Middleware	31
10.5. Database Migrations.....	32
10.6. Environment Configuration	33
10.7. Configuration Files.....	34
10.8. Console Commands and Scheduler	35
10.9. Frontend Assets.....	36
10.10. Authentication System.....	37
10.11. API Logging System.....	38
10.12. Chat Lock Mechanism.....	39
10.13. External API Integration	40
10.14. Dependency and Package Management	41
10.15. Core Framework Entry Files.....	42
11. How the system works (high level)	43
11.1 Main components.....	43
11.2 Data flow	43
11.3 Why artisan commands are used.....	43
12. Project setup (first-time).....	44
12.1 Extract / clone the project.....	44
12.2 Install backend dependencies (Laravel).....	44
12.3 Create environment file and app key.....	44
12.4 Install frontend dependencies (Vite)	44
13. Database setup (SQLite current, MySQL, PostgreSQL).....	45
13.1 SQLite (current).....	45
13.2 MySQL / MariaDB	45
13.3 PostgreSQL	46

13.4 How to create a migration file like that (terminal)	46
14. Configuration (.env) including Sri Lankan time zone.....	47
14.1 Core settings.....	47
14.2 Sri Lankan time zone (Asia/Colombo).....	47
14.3 Chat lock TTL (auto release after 30 minutes).....	47
15. Run the project (step-by-step: 3 terminals)	48
15.1 Terminal 1 – Laravel backend setup.....	48
15.2 Terminal 2 – Frontend assets (Vite).....	48
15.3 Terminal 3 – Scheduler (auto sync).....	48
16. Using the web UI.....	49
17. Syncing messages and contacts (manual + scheduler)	49
17.1 Automatic sync (scheduler).....	49
17.2 Manual sync (CLI).....	49
18. Chat locking (auto + manual release, 30-minute TTL)	50
18.1 Auto release after 30 minutes	50
18.2 Manual release (recommended workflow)	50
19. Admin accounts (seeders + adding new admin via tinkler).....	51
19.1 Default admin accounts (created by seeders)	51
19.2 Add another admin using tinkler.....	51
20. Contacts management	52
20.1 Auto contact sync	52
21. Logs and troubleshooting.....	52
21.1 Where to check logs	52
21.2 Common issues	52
22. Setup on another computer/device (Windows + macOS).....	53
22.1 Checklist (both OS).....	53
22.2 Windows notes.....	53
22.3 macOS notes.....	53

Table of Figures

Figure 1 - Flow8

1. Laravel and MVC Architecture

1.1 Introduction

Laravel is a popular open-source **PHP framework** used for developing modern web applications. It provides a clean and elegant syntax, which makes coding easier and faster for developers. Laravel is widely used because it offers built-in features such as routing, authentication, database handling, and security.

Laravel follows the **MVC architecture**, which helps organize application code in a structured manner. This makes web applications easier to develop, manage, and maintain.

1.2 What is Laravel?

Laravel is a framework designed to simplify web development using PHP. Instead of writing all functionality from the beginning, developers can use Laravel's ready-made tools and features.

Some important features of Laravel include:

- Routing
- Authentication and authorization
- Database migration
- Blade templating engine
- Security features
- API support

Because of these features, Laravel helps developers build secure, scalable, and efficient web applications.

1.3 What is MVC Architecture?

MVC stands for **Model, View, and Controller**. It is a software design pattern used to separate different parts of an application.

1. Model

The Model is responsible for handling data and business logic. It communicates with the database to store, retrieve, update, and delete data.

Example:

A Product model can manage product details in an e-commerce application.

2. View

The View is the presentation layer of the application. It is the part users can see, such as web pages, forms, and buttons.

Example:

A product page displaying product name, price, and description.

3. Controller

The Controller acts as an intermediary between the Model and the View. It receives user requests, processes them, interacts with the model, and returns the appropriate response.

Example:

When a user requests a product page, the controller fetches data from the model and sends it to the view.

1.4 Laravel and MVC Architecture

Laravel is built on the MVC pattern, which makes development more organized.

- **Model** handles database operations
- **View** manages the user interface
- **Controller** processes user requests and application logic

This separation of concerns improves:

- Code readability
- Maintenance
- Reusability
- Team collaboration
- Testing

1.5 Advantages of Laravel with MVC

Using Laravel with MVC architecture provides several benefits:

- **Better code organization**
Each component has its own responsibility.
- **Easy maintenance**
Developers can update one part without affecting others.
- **Faster development**
Built-in Laravel tools save development time.
- **Improved security**
Laravel provides protection against common web threats.
- **Scalability**
Applications can grow more easily with proper structure.

2. Project overview and purpose

SLT WhatsApp Agent is a Laravel-based dashboard to manage WhatsApp customer chats in one place. It supports agent-friendly chat locking (auto lock, manual lock release, and auto release after 30 minutes) to prevent multiple replies to the same customer. Contacts are synced automatically via Laravel Scheduler, with support for manually adding contacts and saving carts/contact persons. The system runs on SQLite by default, with setup options for MySQL and PostgreSQL, and uses the Sri Lankan time zone (Asia/Colombo).

Purpose

- Provide a lightweight web UI to view chats per contact.
- Sync inbound messages from the SLT WhatsApp API into the local database for browsing and auditing.
- Reply to customers from the web UI.
- Keep the contact list updated automatically from "recent active mobiles" (auto contact sync).

3. Key features

- Contact list with last message preview and unread counts.
- Chat view that reads messages from the local database (fast + consistent).
- Sign-in only (self-registration is removed).
- Admin users are created automatically using database seeders.
- Automatic contact normalization (07xxxxxxx → 94xxxxxxx) for consistent storage.
- Auto-sync the most recently active mobile numbers (e.g., last 5) from the SLT endpoint and upsert into contacts.
- Send replies from the chat UI; outbound messages are stored locally for auditing.
- Automatic background sync using Laravel Scheduler (schedule:work).
- Chat locking to prevent multiple admins replying to the same customer at the same time (with auto and manual release).
- API request/response logging for troubleshooting (stored in DB and viewable in UI).

4. Overall System Flow

The overall flow of the system begins when a user accesses the application through a web browser. The request is first handled by Laravel's entry point and routing system. Then the request is directed to the relevant controller, which processes the required business logic. If external communication is needed, the controller uses the SltWhatsappClient service to connect with the SLT WhatsApp API. At the same time, local models interact with the MySQL database to store and retrieve application data.

The overall project flow can be represented as follows:

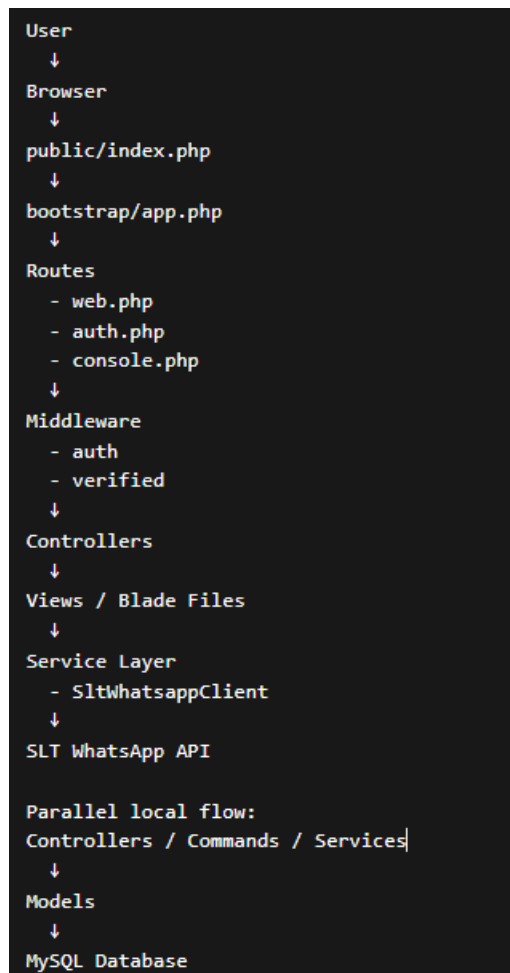


Figure 1 - Flow

5. Requirements

- PHP 8.2+
- Composer (PHP dependency manager)
- Node.js 18+ and npm (for Vite asset building)
- Database: SQLite (current default) OR MySQL/MariaDB OR PostgreSQL
- Git
- Windows: XAMPP/WAMP/Laragon (or standalone PHP)
- macOS: Laravel Herd/Valet or PHP via Homebrew (or standalone PHP)

6. Functional Flow of the System

6.1 Login Flow

When a user accesses the system, the authentication module validates the login credentials using Laravel's auth system. Once authenticated, the user can access the main chat module.

6.2 Chat List Flow

When the user opens the chats section, the system retrieves contact data from the contacts table and displays the chat list.

6.3 Message Retrieval Flow

When a chat is opened, the frontend requests the latest messages for that contact. The controller then calls the SltWhatsappClient service, which fetches the message data from the SLT WhatsApp API. The system may also merge locally stored outgoing messages before displaying them.

6.4 Sending Reply Flow

When the user sends a reply, the request is processed by ChatApiController. The system first checks the chat lock status, identifies the latest relevant message, and then sends the reply through the SLT API. After successful delivery, the message is saved in the local messages table.

6.5 Contact Synchronization Flow

The contact synchronization process is initiated either manually by the user or automatically by the scheduler. The system retrieves recent active mobile numbers from the SLT API and updates the contacts table accordingly.

6.6 Log Monitoring Flow

Every API request made through the service layer is logged into the api_logs table. These logs can later be viewed by administrators through the logs interface.

7. Required Files to Run the Project

The following categories of files are essential for using the project:

Core Framework Files

- artisan
- public/index.php
- bootstrap/app.php
- composer.json

Application Logic Files

- controllers
- models
- services
- console commands

Database Files

- migrations

View Files

- Blade templates in resources/views

Frontend Files

- CSS and JavaScript assets

Configuration Files

- .env
- .env.example
- config files

Dependency Folders

- vendor
- node_modules

8. Controllers of the Project

8.1. ChatController

ChatController is responsible for loading the main chat pages of the system. It mainly handles page rendering and displays the contact list and selected conversation interface for the user.

Main Responsibilities

- loading the chat list page
- opening the selected contact chat
- returning chat-related Blade views

Possible Errors

Contact not found

This error may occur when a user tries to open a contact that does not exist in the database.

Empty contact list

This may happen when no contacts have been synchronized into the database.

Unauthorized access

This error may occur when unauthenticated users attempt to access chat pages.

Solutions

- use route model binding correctly
- display a 404 Not Found page or redirect safely if the contact does not exist
- run the contact synchronization process to load contacts into the database
- display a user-friendly message such as "No contacts available"
- protect routes using auth and verified middleware

8.2. ChatApiController

ChatApiController is the main controller for chat-related operations. It handles retrieving messages, sending replies, and marking chats as read. This controller communicates with both the service layer and the local database.

Main Responsibilities

- retrieving messages for the selected contact
- sending reply messages
- updating read status
- merging external API messages with locally stored messages

Possible Errors

API connection failure

This may occur when the external SLT WhatsApp API is unavailable or unreachable.

Invalid API response

This may happen when the API response format is incomplete, incorrect, or different from the expected structure.

Reply sending failure

This may occur when the message cannot be sent due to invalid UUID values, missing contact details, or API-side failures.

Message duplication

This may happen when locally stored outgoing messages and API messages overlap.

Validation errors

This may occur when users send empty messages or invalid input values.

Solutions

- use exception handling with try-catch blocks
- return proper JSON error responses to the frontend
- log all API failures in the api_logs table
- validate the API response before processing it
- use fallback handling for missing fields
- verify the latest inbound UUID before sending a reply
- validate the contact mobile number before processing
- normalize and merge messages carefully using UUID or timestamp checks
- apply Laravel validation rules before handling requests

8.3. ChatLockController

ChatLockController manages the chat lock mechanism. Its main purpose is to prevent multiple users from replying to the same conversation at the same time.

Main Responsibilities

- acquiring a lock for a chat
- releasing a lock
- checking current lock status

Possible Errors

Lock conflict

This occurs when one user tries to access or reply to a chat that is already locked by another user.

Expired lock not cleared

This may happen when old lock information remains in the system even though it is no longer valid.

Incorrect lock owner identification

This can occur when the system fails to correctly identify which user currently owns the lock.

Solutions

- display a clear warning message when a chat is already locked
- block reply actions until the lock is released
- use lock expiration logic based on the locked_at value
- automatically clear expired locks
- store locked_by_user_id correctly
- verify the current user before allowing lock release or modification

8.4. ContactController

ContactController is responsible for synchronizing and managing contact data. It fetches recent active mobile numbers from the SLT WhatsApp API and stores or updates them in the local database.

Main Responsibilities

- fetching recent active contacts from the API
- inserting or updating local contact records

Possible Errors

Synchronization failure

This may occur when the system cannot retrieve recent contacts from the external API.

Duplicate contact records

This may happen when the same mobile number is inserted multiple times.

Invalid mobile number format

This may occur when the API returns phone numbers in an unexpected format.

Large data volume issues

This may happen when too many contacts are returned at once, causing slower performance.

Solutions

- handle API errors using proper exception handling
- retry failed API requests when necessary
- log synchronization failures for debugging
- use unique validation for mobile numbers
- use updateOrCreate or upsert logic to prevent duplicates
- normalize mobile numbers before saving them
- validate the format of mobile numbers in the code
- apply limits, batching, or pagination for large contact sets

8.5. LogController

LogController is responsible for displaying API log data for monitoring and debugging purposes. It is mainly intended for administrative users.

Main Responsibilities

- retrieving log records
- displaying request and response details
- helping administrators identify system issues

Possible Errors

Too many log records

This may occur when the api_logs table becomes too large, causing slower page loading.

Unauthorized access to logs

This may happen when non-admin users try to open the log page.

Sensitive data exposure

This may occur if request or response logs contain passwords, tokens, or other confidential information.

Solutions

- use pagination when displaying log records
- archive or delete old log records regularly
- restrict access using is_admin checks
- protect log routes with authorization logic
- mask tokens, passwords, and other secrets before saving logs
- avoid storing unnecessary confidential information

8.6. ProfileController

ProfileController handles user profile-related operations such as updating personal information, changing passwords, and deleting user accounts.

Main Responsibilities

- updating profile information
- changing account password
- deleting the user account

Possible Errors

Invalid password update

This may happen when the new password does not meet the required validation rules.

Email update conflict

This may occur when a user attempts to update the email address to one that already exists in the system.

Accidental account deletion

This may happen if the user deletes the account without proper confirmation.

Solutions

- apply Laravel password validation rules
- require password confirmation before updating
- validate email uniqueness before saving changes
- add confirmation steps before deleting an account
- optionally require the current password before destructive actions

8.7. Authentication Controllers

The authentication controllers provided by Laravel manage the user authentication process. These controllers handle login, logout, password reset, and email verification functions.

Main Responsibilities

- logging users in and out
- resetting passwords
- verifying email addresses
- managing authenticated sessions

Possible Errors

Invalid login credentials

This may occur when the user enters an incorrect email address or password.

Email not verified

This may happen when a user tries to access protected routes without verifying the email address.

Session expiration

This may occur when the user session expires while the application is in use.

Solutions

- show proper validation messages for incorrect credentials
- limit repeated login attempts to improve security
- use verified middleware to restrict access
- redirect users to the email verification notice when required
- redirect expired sessions to the login page
- configure session settings correctly in the .env file

8.9. Common Errors in Controllers and Their Solutions

Validation Errors

- Validation errors may occur when user input is incomplete, empty, or incorrectly formatted.

Solutions

- use Laravel form validation
- return clear validation error messages to the user

Database Errors

- Database-related errors may occur due to missing migrations, duplicate records, null values, or schema mismatches.

Solutions

- run migrations correctly before using the system
- define nullable fields where required
- apply database constraints and application-level validation

API Timeout Errors

- API timeout errors may occur when the SLT WhatsApp API responds slowly or becomes temporarily unavailable.

Solutions

- set proper request timeout values
- catch timeout exceptions
- log failed API requests for later analysis

Authorization Errors

- Authorization errors may occur when users attempt to access pages or actions without the required permissions.

Solutions

- apply middleware to protected routes
- check admin privileges where necessary

Unexpected Null Values

- Unexpected null values may occur when required data is missing from either the API response or the local database.

Solutions

- check values before using them in the code
- apply null-safe conditions and fallback handling

9. Models of the Project

9.1. Contact Model

The Contact model represents the WhatsApp contacts stored in the system. This model is used to manage the contact list displayed in the chat interface and also stores chat-related state information such as synchronization details, read status, and locking information.

Main Responsibilities

- storing contact details such as name and mobile number
- maintaining the list of active WhatsApp contacts
- storing synchronization-related information
- keeping track of chat read status
- storing chat lock information for concurrent user handling

Main Data Handled

The Contact model typically handles the following data:

- contact name
- mobile number
- last synchronization time
- last read time
- lock owner user ID
- lock timestamp

Relationships

The Contact model is linked with other parts of the system.

Relationship with Message Model

A contact can have multiple messages associated with it.

Relationship with User Model

A contact may be linked to a user who currently holds the chat lock.

Possible Errors

Duplicate contact records

This may occur when the same mobile number is inserted multiple times during synchronization.

Invalid mobile number format

This may happen when a phone number is stored in an inconsistent or incorrect format.

Missing contact information

This may occur when required fields such as mobile number are empty or null.

Expired lock data remaining in the record

This may happen when old lock information is not cleared properly.

Contact not found

This may occur when the application tries to load a contact that does not exist in the database.

Solutions

- use unique validation or database constraints for the mobile number
- normalize phone numbers before saving them into the database
- validate required fields before creating or updating records
- clear expired lock records using time-based logic
- use route model binding and proper null checking when loading contacts

9.2. Message Model

The Message model represents chat messages related to a contact. In this project, it is mainly used to store local message data, especially outgoing reply messages, and to help merge local records with data received from the external API.

Main Responsibilities

- storing outgoing messages sent by users
- maintaining local message history
- linking messages to contacts
- storing message metadata such as UUID, direction, and sent time
- supporting message display in the chat interface

Main Data Handled

The Message model typically handles:

- contact ID
- message UUID
- message body
- message direction
- sent timestamp
- raw API payload or message data

Relationships

The Message model is connected to the Contact model.

Relationship with Contact Model

Each message belongs to one contact, while one contact can have many messages.

Possible Errors

Duplicate messages

This may occur when the same message is saved multiple times due to repeated synchronization or merging.

Missing message body

This may happen when the text content of the message is empty or null.

Invalid message direction

This may occur when the direction value is not properly defined as incoming or outgoing.

Incorrect message association

This may happen when a message is linked to the wrong contact.

Timestamp inconsistency

This may occur when message time values are not stored or formatted correctly.

Solutions

- use UUID checks or uniqueness logic to prevent duplicate messages
- validate the message body before saving
- define clear allowed values for message direction
- ensure each message is correctly linked using the contact ID
- normalize and validate timestamps before storage

9.3. ApiLog Model

The ApiLog model is used to store API request and response data generated during communication with the SLT WhatsApp API. This model plays an important role in system monitoring, debugging, and performance tracking.

Main Responsibilities

- storing API request details
- storing API response details
- recording endpoint and method information
- tracking response status and execution duration
- supporting debugging and auditing activities

Main Data Handled

The ApiLog model generally stores:

- user ID
- service name
- endpoint name
- request method
- response status
- request payload
- response payload
- execution duration

Relationships

The ApiLog model may be associated with the User model.

Relationship with User Model

A log entry may be linked to the user who initiated the API action.

Possible Errors

Large log data volume

This may occur when too many logs are stored, making retrieval slower.

Sensitive data exposure

This may happen if passwords, tokens, or confidential values are stored in logs.

Incomplete log records

This may occur when request or response data is partially missing.

Incorrect status recording

This may happen when the response status or execution time is not logged properly.

Solutions

- apply pagination when displaying logs
- archive or remove old log entries regularly
- mask sensitive values before saving logs
- use safe logging logic with default fallback values
- validate status and duration fields before storing log records

9.4. User Model

The User model is the default Laravel authentication model used in the project. It represents authenticated system users and administrators who can log into the application and manage chats.

Main Responsibilities

- storing user account information
- managing authentication and login access
- supporting authorization and permission checks
- identifying the user who owns a chat lock
- determining whether a user has admin privileges

Main Data Handled

The User model typically handles:

- user name
- email address
- password
- email verification time
- admin status

Relationships

The User model is connected with different project features.

Relationship with Contact Model

A user may own the lock for one or more contacts.

Relationship with ApiLog Model

A user may be associated with API log records generated by their actions.

Possible Errors

Duplicate email address

This may occur when multiple users are created with the same email.

Invalid password data

This may happen when weak or invalid passwords are accepted.

Unauthorized role access

This may occur when a normal user tries to access admin-only features.

Unverified email usage

This may happen when a user attempts to access protected routes without verifying the email address.

Incorrect lock ownership reference

This may occur when the user ID stored in a lock record is invalid or missing.

Solutions

- enforce unique email validation in both the application and database
- apply strong password validation rules
- use authorization checks and middleware for admin-only actions
- apply verified middleware for protected routes
- validate user references before assigning lock ownership

9.5. Relationships Between Models

Contact and Message

The Contact model has a one-to-many relationship with the Message model. A single contact can have many related messages, while each message belongs to only one contact.

Contact and User

The Contact model can be associated with the User model through the chat lock mechanism. This is used to identify which user is currently handling a contact conversation.

User and ApiLog

The User model can be linked with the ApiLog model to record which authenticated user performed a specific API-related action.

9.6. Common Errors in Models and Their Solutions

Data Duplication

Data duplication may occur when the same contact or message is stored more than once.

Solutions

- use unique constraints in the database
- apply updateOrCreate or upsert logic
- validate records before saving

Missing Required Data

This may occur when important fields such as mobile number, message body, or email are not provided.

Solutions

- apply validation rules before saving records
- use required field checks in controllers and forms

Incorrect Relationships

Relationship-related errors may occur when foreign keys are missing or model links are not defined properly.

Solutions

- define Eloquent relationships clearly
- use proper foreign key values
- validate related records before saving

Invalid Data Format

This may occur when phone numbers, timestamps, UUIDs, or email addresses are stored in inconsistent formats.

Solutions

- normalize data before saving
- apply format validation rules
- use standard date and string handling methods

Database Constraint Errors

These errors may occur when unique values, nullable rules, or foreign key references are violated.

Solutions

- define proper database schema constraints
- align model logic with migration structure
- handle database exceptions gracefully

Performance Issues

Performance issues may occur when the number of contacts, messages, or logs becomes too large.

Solutions

- use pagination for large result sets
- index important fields such as mobile number and foreign keys
- archive unnecessary old records regularly

10. Technical Architecture

10.1. Route Files

The route files are responsible for receiving user requests and directing them to the correct controller methods. They act as the connection point between the user interface and the backend logic of the application.

Main Responsibilities

- defining URL paths of the application
- connecting routes to controllers
- applying middleware such as authentication and email verification
- managing web routes, authentication routes, and console routes

Main Files Used

- routes/web.php
- routes/auth.php
- routes/console.php

Possible Errors

Route not found

This may occur when a requested URL is not defined in the route files.

Middleware access issues

This may happen when users try to access protected routes without authentication or verification.

Wrong controller mapping

This may occur when a route points to an incorrect controller or method.

Solutions

- define all required routes correctly
- protect sensitive routes using auth and verified middleware
- verify controller and method names carefully
- test route access after changes

10.2. Blade View Files

Blade view files are responsible for presenting the user interface of the application. They display chat pages, log pages, authentication pages, and profile pages.

Main Responsibilities

- rendering the frontend interface
- displaying chat conversations and contact lists
- showing forms, navigation, and log details
- supporting dynamic UI updates with backend data

Main Files Used

- resources/views/layouts/app.blade.php
- resources/views/layouts/navigation.blade.php
- resources/views/layouts/guest.blade.php
- resources/views/chats/index.blade.php
- resources/views/chats/show.blade.php
- resources/views/chats/partials/list-items.blade.php
- resources/views/logs/index.blade.php
- resources/views/profile/edit.blade.php
- resources/views/auth/*.blade.php
- resources/views/welcome.blade.php

Possible Errors

View not found

This may occur when Laravel cannot locate the requested Blade file.

Undefined variable in view

This may happen when the controller does not pass required data to the view.

Broken UI rendering

This may occur due to missing partial files or incorrect Blade syntax.

Solutions

- ensure all Blade files exist in the correct locations
- pass required variables from the controller properly
- check Blade syntax carefully
- use partials and layouts consistently

10.3. Service Layer

The service layer contains the business logic for external API communication. In this project, the main service layer file is `SlTWhatsappClient`, which handles all communication with the SLT WhatsApp API.

Main Responsibilities

- authenticating with the external API
- managing bearer tokens
- sending API requests
- retrieving messages and contacts
- sending replies
- logging API activity

Main Files Used

`app/Services/SlTWhatsappClient.php`

Possible Errors

API authentication failure

This may occur when login credentials or tokens are invalid.

API timeout or unavailable service

This may happen when the external API responds slowly or is down.

Invalid response format

This may occur when the API response structure is different from what the system expects.

Solutions

- validate API credentials in the environment file
- use timeout handling and exception catching
- log failed requests for debugging
- validate response data before processing

10.4. Middleware

Middleware is used to filter and control incoming requests before they reach the controller. It provides security and request validation throughout the system.

Main Responsibilities

- checking whether the user is authenticated
- verifying whether the email is confirmed
- protecting restricted routes
- controlling access to important features

Main Middleware Used

- auth
- verified

Possible Errors

Unauthorized access

This may occur when a user tries to open a protected page without logging in.

Email verification restriction

This may happen when a user account is not verified but tries to access secured routes.

Solutions

- apply middleware to all protected routes
- redirect unauthenticated users to the login page
- redirect unverified users to the verification notice

10.5. Database Migrations

Database migrations define the structure of database tables used in the project. They are essential for creating and updating the database schema.

Main Responsibilities

- creating database tables
- adding new columns
- modifying table structure
- supporting version control for the database schema

Main Files Used

- create_users_table
- create_contacts_table
- create_messages_table
- create_api_logs_table
- add_is_admin_to_users_table
- add_chat_lock_to_contacts_table
- add_last_read_at_to_contacts_table

Possible Errors

Migration failure

This may occur when there is a syntax error or schema conflict in migration files.

Missing tables or columns

This may happen when migrations are not run properly.

Foreign key conflicts

This may occur when related records are missing or referenced incorrectly.

Solutions

- check migration syntax carefully
- run php artisan migrate correctly
- define foreign keys properly
- verify schema changes before deployment

10.6. Environment Configuration

The environment configuration contains project-specific settings such as database credentials, application details, and API configuration values.

Main Responsibilities

- storing app-level configuration
- managing database connection settings
- storing external API credentials
- defining session, queue, and cache settings

Main Files Used

.env

.env.example

Possible Errors

Invalid database connection

This may occur when database host, name, username, or password is incorrect.

Missing API credentials

This may happen when SLT API values are not configured.

Incorrect application URL or environment settings

This may cause issues with routing, sessions, or deployment behavior.

Solutions

- configure .env carefully before running the project
- verify database credentials and API settings
- keep .env.example updated as a reference
- clear config cache after changes using Laravel commands

10.7. Configuration Files

Configuration files define project settings for services, database, authentication, chat settings, sessions, and other framework behavior.

Main Responsibilities

centralizing application settings
managing service definitions
defining custom project configuration
supporting maintainability and flexibility

Main Files Used

config/chat.php
config/services.php
config/database.php
config/auth.php
config/session.php
config/cache.php
config/queue.php

Possible Errors

Incorrect configuration values

This may occur when values are missing or wrongly defined.

Service misconfiguration

This may happen when third-party service settings are incomplete.

Cached outdated configuration

This may occur when Laravel continues using old config values after updates.

Solutions

- verify configuration keys and values carefully
- ensure all required settings are present
- clear and rebuild config cache after modifications
- separate custom settings into dedicated config files

10.8. Console Commands and Scheduler

Console commands and the scheduler are used for background and periodic operations. In this project, they are mainly used for synchronizing recent contacts.

Main Responsibilities

- running automated background tasks
- synchronizing recent active contacts
- reducing manual work for admins
- keeping the local system data updated

Main Files Used

- app/Console/Commands/WhatsappSyncContacts.php
- app/Console/Commands/WhatsappSync.php
- routes/console.php

Possible Errors

Scheduled task not running

This may occur when the Laravel scheduler is not configured properly.

Command execution failure

This may happen when the command cannot complete due to API or database issues.

Duplicate synchronization runs

This may occur if the scheduler or manual sync triggers overlap.

Solutions

- configure Laravel scheduler correctly on the server
- log command failures for debugging
- handle exceptions inside command logic
- add safe checks to avoid repeated overlapping sync tasks

10.9. Frontend Assets

Frontend assets include the CSS, JavaScript, and build configuration files used to support the user interface.

Main Responsibilities

- styling the user interface
- adding frontend interactivity
- compiling and managing frontend resources
- supporting responsive and consistent layout behavior

Main Files Used

- resources/js/app.js
- resources/css/app.css
- vite.config.js
- tailwind.config.js
- package.json

Possible Errors

Styles not loading

This may occur when CSS assets are not compiled correctly.

JavaScript functions not working

This may happen when frontend scripts fail to load or contain errors.

Build errors

This may occur when Vite or package dependencies are not installed correctly.

Solutions

- install frontend dependencies properly
- compile assets using Vite commands
- check JavaScript console errors
- verify file paths and import statements

10.10. Authentication System

The authentication system is a core part of the application that manages login, logout, password reset, and user session handling.

Main Responsibilities

- logging users into the system
- maintaining secure sessions
- allowing password reset and email verification
- restricting access to authorized users only

Main Files Used

- authentication controllers
- authentication Blade files
- routes/auth.php
- app/Models/User.php

Possible Errors

Login failure

This may occur when credentials are invalid.

Session expiration

This may happen when the user session times out while using the application.

Access to protected pages without authentication

This may occur when routes are not properly secured.

Solutions

- validate login credentials properly
- configure session settings correctly
- protect routes using authentication middleware
- provide clear user feedback during login errors

10.11. API Logging System

The API logging system records request and response information related to communication with the SLT WhatsApp API.

Main Responsibilities

- storing API activity
- tracking endpoint responses
- helping with debugging
- supporting monitoring and auditing

Main Files Used

- app/Models/ApiLog.php
- LogController
- api_logs table

Possible Errors

Missing logs

This may occur when failed requests are not saved properly.

Excessive log storage

This may happen when the number of logs becomes too large.

Sensitive information in logs

This may occur when confidential data is stored without masking.

Solutions

- ensure all API requests pass through logging logic
- archive or delete old logs regularly
- mask sensitive data before storing log entries
- use pagination to display logs efficiently

10.12. Chat Lock Mechanism

The chat lock mechanism is an important project feature that controls concurrent access to the same chat by multiple users.

Main Responsibilities

- preventing multiple users from replying to the same chat at once
- identifying the user currently handling a chat
- releasing expired or inactive locks

Main Files Used

- ChatLockController
- Contact model
- contacts table fields related to locking

Possible Errors

Lock conflict between users

This may occur when multiple users attempt to access the same chat simultaneously.

Expired lock remains active

This may happen when old lock data is not cleared.

Invalid lock ownership reference

This may occur when the lock owner record is incorrect or missing.

Solutions

- use lock expiration logic
- verify lock ownership before updates
- display user-friendly lock status messages
- clear expired locks automatically

10.13. External API Integration

External API integration is the main communication bridge between the Laravel application and the SLT WhatsApp platform.

Main Responsibilities

- retrieving recent active contacts
- fetching chat messages
- sending reply messages
- authenticating requests to the external platform

Main Files Used

- app/Services/Sl WhatsAppClient.php
- environment variables related to SLT API
- controllers that call the service layer

Possible Errors

API server unavailability

This may occur when the external platform is temporarily down.

Authentication token expiration

This may happen when the bearer token becomes invalid.

Unexpected data structure

This may occur when the API returns data that the system cannot process correctly.

Solutions

- retry authentication when tokens expire
- implement proper exception handling
- validate all incoming response structures
- record failures in logs for later troubleshooting

10.14. Dependency and Package Management

Dependency and package management is required to run the Laravel backend and the frontend build tools correctly.

Main Responsibilities

- managing PHP dependencies
- managing Node.js dependencies
- supporting framework features and asset building

Main Files Used

- composer.json
- package.json
- vendor/
- node_modules/

Possible Errors

Missing dependency packages

This may occur when required packages are not installed.

Version conflicts

This may happen when package versions are incompatible.

Autoloading errors

This may occur when Composer autoload files are not generated correctly.

Solutions

- run dependency installation commands correctly
- use compatible package versions
- regenerate autoload files when needed
- keep dependency files updated and consistent

10.15. Core Framework Entry Files

These files are responsible for starting the Laravel application and preparing the request lifecycle.

Main Responsibilities

- bootstrapping the application
- handling incoming requests
- loading framework services and routes

Main Files Used

- public/index.php
- bootstrap/app.php
- artisan

Possible Errors

Application bootstrap failure

This may occur when framework files are missing or configuration is broken.

Server entry point issue

This may happen when the web server is not correctly pointed to the public directory.

Command-line execution issue

This may occur when artisan commands fail due to incorrect setup.

Solutions

- ensure Laravel core files are present and unchanged
- configure the server document root correctly
- verify PHP and framework dependencies
- test artisan commands after project setup

11. How the system works (high level)

11.1 Main components

- Web UI (Blade templates + Vite): Shows contacts, messages, and send reply UI.
- Database: Stores contacts, messages, users/admins, logs, and other Laravel tables (sessions, cache, etc. depending on config).
- SLT WhatsApp API client/service: Handles login/token, fetches messages, fetches recent mobiles, and sends replies.
- Artisan commands: Dedicated CLI commands perform sync work (messages + contacts).
- Scheduler: Runs the sync commands periodically when enabled (php artisan schedule:work).

11.2 Data flow

1. Contacts exist in the contacts table (created manually or via auto-sync).
2. A sync command pulls messages for contacts and inserts any new ones into messages.
3. The UI reads messages from the DB (not directly from the SLT API), so browsing is fast and consistent.
4. When an admin replies, the app calls the SLT reply endpoint and also stores an outbound message row locally.

11.3 Why artisan commands are used

Syncing is separated into artisan commands so the same logic can run:

- Automatically (via the scheduler) and
- Manually (developer/admin can run when needed)

This keeps web requests fast and makes it easy to run background updates reliably.

12. Project setup (first-time)

12.1 Extract / clone the project

Place the project in a folder like:

- Windows: C:\Projects\slt-whatsapp-demo
- macOS: ~/Projects/slt-whatsapp-demo

12.2 Install backend dependencies (Laravel)

composer install

Why: Downloads Laravel and required vendor libraries into the vendor/ folder.

12.3 Create environment file and app key

Create .env from .env.example:

cp .env.example .env

Windows PowerShell equivalent:

Copy-Item .env.example .env

Generate app encryption key:

php artisan key:generate

Why: APP_KEY is required for sessions, encryption, and security features.

12.4 Install frontend dependencies (Vite)

npm install

Why: Installs Vite and frontend dependencies used to compile CSS/JS assets.

13. Database setup (SQLite current, MySQL, PostgreSQL)

Current default (development)

This project is currently configured to use SQLite for quick local development.

You can switch to MySQL or PostgreSQL for team/shared environments or closer-to-production behavior.

13.1 SQLite (current)

1) Create the SQLite file (if it does not exist):

```
touch database/database.sqlite
```

2) Set the following in .env:

```
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/project/database/database.sqlite
```

3) Run migrations and seeders:

php artisan migrate --seed

13.2 MySQL / MariaDB

1) Create a database (example: slt_whatsapp_demo).

2) Update .env with your DB credentials.

3) Run migrations and seeders.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=slt_whatsapp_demo
DB_USERNAME=root
DB_PASSWORD=your_password
```

php artisan migrate --seed

Notes:

- Make sure the MySQL service is running.
- If you use XAMPP/Laragon, create the DB in phpMyAdmin or via MySQL CLI.

13.3 PostgreSQL

- 1) Create a database (example: slt_whatsapp_demo).
- 2) Update .env with your DB credentials.
- 3) Run migrations and seeders.

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=slt_whatsapp_demo
DB_USERNAME=postgres
DB_PASSWORD=your_password
```

php artisan migrate --seed

Notes:

- Ensure the PostgreSQL service is running.
- On some systems you may need to create the database user and grant privileges.

13.4 How to create a migration file like that (terminal)

How to generate that file:

- **php artisan make:migration create_messages_table --create=messages**

Then edit the generated file in database/migrations/... and run:

- **php artisan migrate**

14. Configuration (.env) including Sri Lankan time zone

14.1 Core settings

Commonly used values:

```
APP_URL=http://localhost:8000
APP_ENV=local
APP_DEBUG=true
```

14.2 Sri Lankan time zone (Asia/Colombo)

To ensure timestamps (locks, logs, message times) use Sri Lankan time:

- 1) Set APP_TIMEZONE in .env
- 2) Ensure config/app.php reads APP_TIMEZONE (Laravel default does).

```
APP_TIMEZONE=Asia/Colombo
```

After changing env/config values, clear config cache if needed:

```
php artisan config:clear
php artisan cache:clear
```

14.3 Chat lock TTL (auto release after 30 minutes)

Chat locks prevent multiple admins from replying at the same time.

The lock automatically expires after 30 minutes (1800 seconds) unless released earlier.

```
CHAT_LOCK_TTL_SECONDS=1800
```


15. Run the project (step-by-step: 3 terminals)

15.1 Terminal 1 – Laravel backend setup

Open the VS Code terminal and run:

```
composer install  
php artisan key:generate  
php artisan migrate --seed  
php artisan serve
```

Keep this terminal running.

Open: **http://localhost:8000**

15.2 Terminal 2 – Frontend assets (Vite)

Open a new terminal in VS Code and run:

```
npm install  
npm run dev
```

Keep this terminal running while developing so assets stay updated.

15.3 Terminal 3 – Scheduler (auto sync)

Open a new terminal in VS Code and run:

```
php artisan schedule:work
```

This runs Laravel Scheduler continuously.

It is required for automatic background sync tasks (such as periodic message sync).

16. Using the web UI

- Log in with an admin account created by seeders (see Section 12).
- Go to the Chats page and select a contact on the left.
- View inbound/outbound messages in the conversation view.
- Type a message and send a reply (subject to chat lock rules).
- Use Sync controls (if available) to pull latest messages/contacts manually.

17. Syncing messages and contacts (manual + scheduler)

17.1 Automatic sync (scheduler)

When `php artisan schedule:work` is running, the app can execute:

- Message sync command (example: `whatsapp:sync`)
- Contact sync command (example: `whatsapp:sync-contacts`)

These tasks run on a schedule (commonly every minute) as configured in the project console scheduler.

17.2 Manual sync (CLI)

You can run these manually for debugging or on-demand:

```
php artisan whatsapp:sync --limit=30
```

```
php artisan whatsapp:sync --contact_id=1 --limit=30
```

```
php artisan whatsapp:sync-contacts --limit=5
```

18. Chat locking (auto + manual release, 30-minute TTL)

Chat locking is a safety mechanism for multi-admin usage to prevent two admins replying to the same customer simultaneously.

Lock storage (typical)

- stored per-contact (e.g., locked_by_user_id, locked_at)

Lock behaviors

- Auto lock: When an admin opens/starts interacting with a chat (or attempts to send), the system can claim a lock for that admin.
- Auto release: Locks automatically expire after 30 minutes (CHAT_LOCK_TTL_SECONDS=1800). This handles cases like browser crash or admin leaving without releasing.
- Manual lock/release: An admin can manually release a lock (for example, via a "Release" action in the chat UI, if enabled).

18.1 Auto release after 30 minutes

If the lock is older than 30 minutes, it is treated as stale and the system will clear it automatically on the next lock check.

This prevents long-lived dead locks.

18.2 Manual release (recommended workflow)

- If you finish a conversation, use the UI's release option (if available) to free the chat immediately.
- If you need to hand off a chat to another admin, manually release so the other admin can respond right away.

19. Admin accounts (seeders + adding new admin via tinker)

19.1 Default admin accounts (created by seeders)

Admin users are currently created using database seeders when you run:

- **php artisan migrate --seed**

The following admin accounts are created when you run database seeding:

Admin Name	Email	Default Password
Kosala	kosala@slt.lk	Admin@123
Charith	charith@slt.lk	Admin@123
Menusha	menusha@slt.lk	Admin@123
Ekvith	ekvith@slt.lk	Admin@123

After first login, change passwords as required by your security policy.

19.2 Add another admin using tinker

Run:

php artisan tinker

Then create a new admin user (example):

```
\App\Models\User::create([
    'name' => 'New Admin',
    'email' => 'newadmin@slt.lk',
    'password' => bcrypt('ChangeMe@123'),
    'email_verified_at' => now(),
]);
```

Exit tinker:

Exit

20. Contacts management

20.1 Auto contact sync

The scheduler (or manual artisan command) fetches the most recently active mobile numbers from the SLT endpoint and upserts them into the contact's table.

Normalization example:

- 07xxxxxxx → 94xxxxxxx

This keeps the agent/admin contact list fresh without manual entry.

21. Logs and troubleshooting

21.1 Where to check logs

- storage/logs/laravel.log
- Logs page in the UI (API call log entries, if available)

21.2 Common issues

- Scheduler not running: keep `php artisan schedule:work` running in Terminal 3.
- Frontend unstyled: run `npm install` and keep `npm run dev` running in Terminal 2.
- DB errors: verify `DB_CONNECTION`/credentials and rerun **`php artisan migrate --seed`**.
- Unauthorized API calls: verify SLT API credentials/token variables in `.env`.

22. Setup on another computer/device (Windows + macOS)

22.1 Checklist (both OS)

- Install PHP 8.2+, Composer, Node.js (npm).
- Copy the project folder (zip/unzip) or git clone.
- Create .env and set DB + SLT API variables; set APP_TIMEZONE=Asia/Colombo.
- composer install
- npm install
- Set up database (SQLite/MySQL/PostgreSQL) and run php artisan migrate --seed
- Run the 3 terminals as described in Section 8.

22.2 Windows notes

- If using XAMPP/Laragon, ensure PHP 8.2+ is active and available in PATH.
- Use phpMyAdmin or CLI to create MySQL database if using MySQL.
- Make sure you run commands from the project root folder in VS Code terminal.

22.3 macOS notes

- Recommended: Laravel Herd or Homebrew PHP (brew install php@8.2).
- MySQL/PostgreSQL can be installed via Homebrew or Docker.
- Ensure storage/ and bootstrap/cache are writable if running under a web server.